PROXMOX AUTOMATION SERIES · PHASE 5

The Gate and the Watchman

Admission control that refuses before the clone spends anything, a health audit that remembers what live state forgets, and the queue that turned out to be a mutex. Every branch field-proven on 30 August 2026.

PVE 9.2 · SEMAPHORE UI 2.19.11 · ANSIBLE-CORE 2.21
COMMUNITY.PROXMOX 2.0.0 · PROXMOXER 2.3.0 · MSMTP
AUGUST 2026 · COMPANION REPO: PROXMOX-AUTOMATION

ADMIT · AUDIT · REMEMBER

Contents

1. What Phase 5 Adds

This is the sixth module of the Proxmox Automation Series, and the one that closes the roadmap. Phases 1 through 4 built the machine: a golden template, serial-tagged storage, the agent layer, and a typed request form on Semaphore that turns one submission into a delivery-checked VM with a receipt. Phase 5 is what the machine was still missing when real customers started asking for things: a gate in front of the form that can say no, a watchman that walks the lab twice a day whether anyone is watching or not, and an honest answer to the question every shared pipeline eventually faces, which is what happens when two people click at once.

The roadmap row, as it stood when Phase 4 closed, read: registry, IP guard, drift audit, parallelism stress test, with NetBox as an optional last resort. Like most of the series, the row survived contact with the field in shape but not in detail: the registry never shipped, NetBox stayed unnecessary, and two of the three legs ended up answering questions nobody had put on the list. This guide tells all three stories the series way, with the commands and their outputs, the prediction scoreboard including its misses, and the errors kept on the page where they happened.

The four asks, scored against the repo

Phase 5 began with a chat log, not a plan. A friend of the operator, reviewing the project on 28 August, sent four concrete feature asks from a customer's point of view. The exercise that followed is one this guide recommends to anyone finishing a pipeline: score each ask against what the repo already does, before building anything. The scoreboard, reproduced exactly as it was filed, is Table 1.

The ask (as phrased in the chat)	Status that morning	Where it landed
Do not let anyone order 768 MB of RAM	ALREADY DONE, twice over	vm_vcpu/vm_ram_gb are enum survey fields (the form can only offer what the pipeline allows), and play 00 re-asserts every value server-side from vm_allowed_vcpu/vm_allowed_ram_gb because --extra-vars and the API bypass the form. The defense-in-depth pattern was in place since Phase 4.
When capacity runs out, refuse to create the VM	NOT DONE	00 had only the per-request node cap vm_max_vcpu (finding #12); there was no aggregate capacity check at all. This became Phase 5a, the admission gate.
Let the customer see their subscription status	NOT DONE, and structurally out of scope	Semaphore is a task runner, not a portal; it cannot host a customer screen. Explored as a separate portal (this chapter), then dropped by operator decision. The surviving piece, a periodic report of fleet and budget, lives in the 5b digest.
Morning and evening Proxmox health check, plus mail	NOT DONE, but trivial	A scheduled playbook plus an SMTP relay, overlapping the planned drift audit exactly. This became Phase 5b, the health audit.

Table 1. The four asks of 28 August 2026, scored against the repository before anything was built. Two were already done or cheap; only two needed new code.

The scoreboard reframed the phase immediately. The quota ask and the health ask were real work with a home in the existing playbooks. The subscription screen was the interesting one: it is a product feature, not a pipeline feature, and chasing it would have meant building a second face for the machine. That fork got its own short investigation before it was dropped, and the investigation is worth keeping because the conclusion is the reason the rest of this guide is UI-free.

The portal that was not built

The first question was architectural: can you put a customer portal in front of Semaphore at all, or does the UI own the house? The answer, verified against how the product actually ships, is that there is nothing to work around. Semaphore is one Go binary listening on one port (3000) serving both the web UI and the REST API; the UI is an embedded single-page application that itself consumes that same API. The API is a first-class interface. A portal would authenticate once with an API token (Authorization: Bearer, created in user settings), and its entire surface would be three calls: start a task with survey parameters, list tasks with statuses, read a task's output. The order button, the orders feed, the green-or-red detail page; a portal is literally a face over three curls.

The second question was which framework, and the honest answer came from listing what a custom executor would have to rebuild before it matched what Semaphore already does silently: the job queue and its serialization, secrets custody for the cluster admin token, the audit trail, git pull-before-run with

the deploy key, output streaming, scheduling. Six systems, all already running. On the framework axis the shortlist came down to Django over Flask for a customer-facing portal, because auth, forms-with-enums, and an admin screens module come free, and the stack estimate landed at roughly 600 lines: Django, SQLite, one Semaphore token, one read-only Proxmox token for the my-VMs screen, no Celery, no Redis, no npm.

Then the operator closed the folder. The verdict, in his own shorthand, was to drop it: the customer asks that survived were the ones the pipeline itself could answer, and a second face meant a second thing to own. It was the same reflex that had retired the custom Angular plan in Phase 4, applied one level higher: Semaphore stays the only face, and Phase 5 ships as playbooks and schedules. What survived the fold: the enum limits (already done), the capacity gate (5a), the health mail (5b), and the subscription screen folded into the 5b digest as a periodic fleet and budget report. The portal design was not wasted; it is parked in the project memory with its API surface mapped, so if the customer ask ever returns, it starts from three curls, not from a blank page.

3

asks survived the fold: the capacity gate became 5a, the health mail became 5b, and the subscription screen became a section of the 5b digest. Nothing else from the portal track shipped, and nothing else was needed.

What shipped instead

Leg	What it is	Where it lives	How it closed
5a, the gate	Admission control: IP collision check against every ipconfig0, a TCP/22 liveness probe for machines the API cannot see, and an allocated-capacity budget. A refusal happens before the clone and creates nothing.	00_provision_vm.yml, section 2b; knobs capacity_max_vcpu/capacity_max_ram_gb in vars/lab-environment.yml	Field-proven 4/4 branches on 30 Aug 2026: one admission grant with live math, three refusals, each exact to the character (chapter 3).
5b, the watchman	A read-only health audit: cluster quorum, node pressure, fleet budget, the managed fleet, drift against its own baseline, guest services and filesystems over SSH; a digest printed to the task log always and mailed when the runner is armed; a verdict that goes red only when a human should read it.	08_health_audit.yml + templates/audit-mail.j2 + scripts/sem-01-msmtp.sh; knobs in vars/lab-environment.yml	Field-proven end to end 30 Aug 2026: baseline write, drift-zero, mailed digest in the inbox, and a scheduler-driven fire (chapter 5, chapter 6).
5c, the accident	The concurrency question the roadmap called a stress test: does the admission gate race when two requests arrive together?	No new code; the finding lives in semaphore/README.md and the README roadmap	Closed by observation 30 Aug 2026: the Semaphore task queue serializes everything, so the race is unreachable from the UI. The deleted-VM alarm and its self-heal were proven in the same window (chapter 7).

Table 2. Phase 5 as shipped: three legs, three different shapes. One is code in front of the clone, one is code on a schedule, one is a field finding that retired a plan.

Figure 1. The gate and the watchman: the two loops Phase 5 added around the Phase 4 pipeline. The gate runs per request and refuses before spending anything; the watchman runs on a schedule and remembers what live state forgets. The queue that serializes both is itself a field finding (chapter 7).

One privilege this volume enjoys over its predecessors: freshness. The guide series had to be rebuilt once after a rollback ate a build pipeline, and two volumes were written days after their field work from memory records. This one is being written with the worklog still warm; every transcript below was pasted by the operator within minutes of the run it describes, and the repo state it documents is the state that produced them.

2. The Gate: Admission Control in Play 00

Before Phase 5, the pipeline's limits were all per-request: the survey enums, the server-side re-assertion of every value, and vm_max_vcpu, the node's start-time cap from finding #12. Nothing stopped ten well-formed requests from collectively promising the node more than it has. Phase 5a is the missing aggregate: a fleet gate that runs after the form has been validated and before the clone spends anything.

Live state, not a registry

The roadmap said registry: record every VM in a file, check the file. The design that shipped says the opposite, and the argument is short. Both questions the gate needs answered, who owns this IP and how much is allocated, are answerable from live cluster state: every VM's ipconfig0 and config.cores/memory sit in the API right now. A registry would be a second thing that can drift, and the lab already deletes VMs out-of-pipeline, which is exactly the habit that rots registries. The API is the truth; the gate reads it and keeps nothing.

The decision was grounded in module source, not documentation folklore, because the numbers had to be right. community.proxmox 2.0.0's proxmox_vm_info merges two API endpoints: the VM list comes from /cluster/resources, which carries vmid, name, and template as a real boolean, and each VM's config comes from /nodes/N/qemu/V/config?current=1, whose values arrive as strings. One consequence decides the budget math: the cluster view reports maxmem 0 for a STOPPED VM, so a runtime view cannot budget; the gate must sum config.memory, the allocation PVE owes the VM whether it runs or not.

The three questions, as one assert

The gate asks three questions and refuses on the first that fails. Does another VM already own the requested IP in its ipconfig0? Is something alive at that IP even though no VM claims it, a manual box, a DHCP lease, an LXC, anything with sshd? Would admitting this VM push the fleet's allocated vCPU or RAM past the budget? The shipped code, exactly as it runs today, is compact enough to read in full:

```
00_PROVISION_VM.YML, SECTION 2B, THE DECISION (VERBATIM)
```

```
- name: Admission control - refuse before the clone spends anything
vars:
ip_collision: >-
{{ ip_records | default([])
| selectattr('ip', 'equalto', vm_ip)
| rejectattr('vmid', 'equalto', target_vmid | int) | list }}
ip_owned_by_target: >-
{{ ip_records | default([])
| selectattr('ip', 'equalto', vm_ip)
| selectattr('vmid', 'equalto', target_vmid | int)
| list | length > 0 }}
ansible.builtin.assert:
that:
- ip_collision | length == 0
- ip_probe.failed | default(false) | bool or ip_owned_by_target | bool
- allocated_vcpu | int + spec_vcpu | int <= capacity_max_vcpu | int
- allocated_ram_mb | int + spec_ram_gb | int * 1024 <= capacity_max_ram_gb | int *
1024
success_msg: >-
Admission granted: {{ vm_ip }} is free (or ours), fleet after
this VM {{ allocated_vcpu | int + spec_vcpu | int }}/{{ capacity_max_vcpu }}
vCPU, {{ (allocated_ram_mb | int + spec_ram_gb | int * 1024) // 1024 }}/{{
capacity_max_ram_gb }} GB.
fail_msg: >-
Admission refused (nothing was created): {{ vm_ip }} -
{% if ip_collision | length > 0 %}already assigned to VMID
{{ ip_collision[0].vmid }} ({{ ip_collision[0].name }});
pick another IP or retire that VM first{% elif not (ip_probe.failed | default(false) |
bool) %}something
is ALIVE there (TCP/22 answered) but no VM holds it in ipconfig0
- a manual or DHCP machine squats that address; pick another IP{% else %}budget
exceeded: adding {{ spec_vcpu }} vCPU / {{ spec_ram_gb }} GB would take the
fleet to {{ allocated_vcpu | int + spec_vcpu | int }}/{{ capacity_max_vcpu }} vCPU and
{{ (allocated_ram_mb | int + spec_ram_gb | int * 1024) // 1024 }}/{{
capacity_max_ram_gb }} GB
allocated; lower the request or decommission a VM (see
vars/lab-environment.yml capacity_max_*){% endif %}.
```

Four conditions, and each earns its line. The first is the collision: any VM other than the adoption target holding the IP refuses the request, naming the owner and its VMID, because a message that names the culprit is a message that ends the conversation. The second is the probe condition, and its shape is deliberate: either nothing answered on TCP/22 (a fresh IP, the good outcome for a new request), or the machine that answered is the target itself (adoption, legal by definition). Anything else, something alive that no VM claims, is a squatter and refused. The third and fourth are the budget, and the boundary is inclusive on purpose: a fleet at exactly 8/8 vCPU is full, not over.

The fail message routes by priority, most actionable first: collision outranks squatter outranks budget. The field would prove this routing twice, once by accident (chapter 3), when a misfiled test form produced both a collision and a live probe answer, and the assert correctly reported the collision as THE failed assertion because fixing it makes the others moot.

Five tasks feeding one decision

The assert's inputs come from five tasks above it. The cluster read is a proxmox_vm_info over all qemu VMs with config: current. The IP records task walks that list and pulls each ipconfig0 apart with a regular expression, ip=([0-9.]{7,15}), building one record per VM that carries a static IP. The sums task walks the same list accumulating cores times sockets and memory, skipping templates and skipping the adoption target, so a re-run never counts the same VM twice and never fails on its own IP. The probe is a wait_for on port 22 with a two-second timeout and ignore_errors, because for a fresh IP the failure is the answer; and the placement of all of it is between the hostname existence lookup and the clone, so a refusal creates nothing, no half-built VM, no cleanup, no orphans.

THE IP RECORDS TASK, AND THE COMMENT THAT EARNS ITS LINES (VERBATIM)

```
- name: Collect the static IP records (ipconfig0 of every VM)
# ipconfig0 is the only interface this pipeline configures, so it is
# the only place a pipeline-style collision can hide. DHCP/manual
# machines are invisible here - that is exactly what the liveness
# probe below is for. Loop+append, not a comprehension: dict/list
# literals inside a Jinja comprehension break the parser (see the
# disk-records task above for the field note).
ansible.builtin.set_fact:
ip_records: >-
{{ ip_records | default([])
+ [{'ip': (item.config.ipconfig0 | default('')
| regex_findall('ip=([0-9.]{7,15})'))[0],
'vmid': item.vmid | int,
'name': item.name | default('vmid-' ~ item.vmid)}] }}
when: (item.config.ipconfig0 | default('')
| regex_findall('ip=([0-9.]{7,15})')) | length > 0
loop: "{{ cluster_vms.proxmox_vms }}"
```

Decision	The alternative	Why this one
Live state, not a registry	a fleet file, or NetBox	the API cannot drift from itself; the lab deletes VMs out-of-pipeline, which is precisely the habit that rots a registry. The intent record moved to 5b, where comparing intent against live is the actual job.
wait_for on port 22, not ping	an ICMP probe	wait_for is a pure Python socket, no iputils-ping dependency in an LXC runner; and port 22 is what the pipeline actually needs reachable anyway.
Allocated, not used	runtime consumption	PVE owes config.memory whether the VM runs or not; the cluster view reads maxmem 0 for stopped VMs, so a runtime view cannot budget.
qemu VMs only, templates excluded	every object on the node	templates and CTs are infrastructure, not fleet budget; CTs (ctrl-01, sem-01) run the pipeline itself and keep their own headroom.
Target excluded from its own sums	count everything	an adoption re-run would otherwise double-count the target and refuse its own IP; with the exclusion, adoption never fails on facts it created.
Refusal before the clone	create, then verify	a rejected request leaves nothing behind: changed=0, a clean qm list, and a red Semaphore task that doubles as the receipt of the refusal.

Table 3. The design decisions behind section 2b, each with the alternative it beat. All six were encoded as README design-decision rows the day they shipped.

The sandbox harness, and its one catch

The gate never went to the field unproven. A simulation harness copied the section 2b expressions verbatim into a scenario playbook, with fixtures shaped exactly per the module source (string config values, boolean template flags), and ran twelve scenarios: adoption self, fresh collision, squatter, stopped-owner-still-counts, IP change, IP steal, DHCP-invisible, vCPU budget, RAM budget, boundary equality, empty records, and the exclusion pins. Result: 12 of 12 pass, with the exclusion sums pinned so a regression in template or target exclusion cannot hide. The harness caught one real bug before the field, a loop_var collision where the outer scenario item shadowed the inner fleet item, which is the second time in the series a verbatim-copy harness paid for itself before the first real run. The knobs the gate reads live in the environment file with their scope documented inline:

VARS/LAB-ENVIRONMENT.YML, THE CAPACITY BLOCK (VERBATIM)

```
# ---- Phase 5a: the fleet admission budget --------------------------------
# The admission gates in 00_provision_vm.yml (section 2b) refuse a request
# BEFORE the clone when adding it would push the ALLOCATED (not used)
# vCPU/RAM of all non-template qemu VMs past these numbers. Deliberately
# below the node's physical ceiling: the budget leaves headroom for the
# CTs (ctrl-01, sem-01) and the laptop host itself. Tune to your hardware;
# a refusal prints the full math, so a wrong budget announces itself.
capacity_max_vcpu: 8
capacity_max_ram_gb: 16
```

Note the last comment line, because it is a design stance: a wrong budget is not a silent failure. Every refusal prints the full arithmetic, fleet after admission against the cap, so an operator who sets 8 vCPU on a node that cannot serve 6 finds out from the first refusal message, not from a monitoring dashboard three weeks later. The gate refuses loudly and explains itself; that is all it was asked to do.

3. The Gate, Field-Proven

The field runbook for 5a had four tests, escalating by branch: A, an adoption re-run that should be admitted with live math; B, a fresh request on an owned IP that should be refused with the owner named; C, a request on the one lab address that is alive but unclaimed, the runner's own IP, which should trigger the squatter refusal; D, a 16 GB request against a nearly full budget that should be refused on arithmetic. Before any of them ran, the field taught its first lesson for free.

Run #38: the gate that was not there

The first run after the 5a code was written came back green, and proved nothing. Task #38, an adoption re-run of the previous form, completed the whole pipeline to the delivery receipt, localhost ok=16 changed=2 skipped=2, vm-faz4-01 ok=86, and not one line of it was new code. Three independent proofs, any one of which would have settled it: the git pull line in the task log read a47dd9c..0578af7, and 0578af7 is the pre-5a head; the task arithmetic matched the pre-5a playbook exactly, 18 localhost task slots with adoption skipping 2, where the 5a version has 23 slots and would have read ok=21; and no admission task or success message appeared anywhere in the output.

RUN #38, THE PULL LINE THAT TOLD THE TRUTH (AS PASTED)

```
TASK [Prepare repository] ************************************
git pull
Already up to date.
a47dd9c..0578af7 main -> origin/main

# 0578af7 = the Phase-4-final head. The 5a push would end at a
# NEW hash above it. Nothing after this line matters: the task
# ran last week's code, green, and proved nothing about 5a.
```

The root cause was procedural, not technical: the zip had not been extracted, committed, and pushed on ctrl-01 before the form was fired. Semaphore tasks clone the bare repo on ctrl-01, so uncommitted edits, or committed-but-unpushed edits, are indistinguishable from no edits at all. The lesson earned a slot in this guide because it is the class of mistake that looks like success: a green run of old code is the most convincing non-proof there is. The fix was the boring one, extract, verify git status shows exactly the expected files, commit, push, verify the new hash, and Test A was re-run against commit 716507d.

Test A: admission granted, with a correction

Run #41 executed the new code and admitted the adoption with live arithmetic the prediction had gotten wrong. The prediction, computed from repo history, said the fleet after admission would read 4/8 vCPU and 8/16 GB, assuming vm-faz4-01 was the only non-template VM. The live cluster said 5/8 and 7/16, because three alpine VMs nobody had mentioned in months were sitting on the node holding 3 vCPU and 3 GB between them. The gate read live state and was right where the prediction was wrong, which is the strongest argument the design has: not the assert passing, the assert correcting us.

RUN #41, THE ADMISSION LINE AND THE RECEIPT (AS PASTED, CONDENSED)

```
TASK [Admission control - refuse before the clone spends anything] ***
ok: [localhost] => {
"assertion": "ip_collision | length == 0",
"changed": false,
"msg": "Admission granted: 192.168.122.60 is free (or ours),
fleet after this VM 5/8 vCPU, 7/16 GB."
}

TASK [Print the delivery receipt] ****************************
ok: [localhost] => {
"msg": {
... "Fleet after admission: 5/8 vCPU, 7/16 GB allocated", ...
}
}

PLAY RECAP ***************************************************
localhost : ok=21 changed=2 skipped=2 failed=0
vm-faz4-01 : ok=86 changed=2 failed=0

# ok=21 skipped=2 = the 5a adoption arithmetic (pre-5a read ok=16);
# the 3 alpines are the difference between the predicted 4/8+8/16
# and the live 5/8+7/16.
```

The run also settled the exclusion mechanics on real data: the IP records task found exactly one ipconfig0 carrier, vm-faz4-01, the alpines and the template carrying none; and the budget task's loop labels showed vm-faz4-01 skipped as the target and ubuntu-2604-tpl skipped as template, while the alpines were summed. Adoption re-runs from that day on printed the same line with the same shape, and the numbers would be cross-reported by the audit playbook later the same day: three independent code paths reading one truth.

Test B: the collision, naming its victim

Test B asked for a fresh VM at 192.168.122.60, the IP vm-faz4-01 already holds in ipconfig0. The refusal came back within seconds, before any clone, naming the owner exactly as designed, and the recap arithmetic is the receipt of a clean refusal: fourteen tasks ran, zero changed anything, one failed on purpose. The qm list pasted alongside it showed no trace of vm-collide-test anywhere. Nothing was created; nothing needed cleanup.

TEST B, 08:12, THE REFUSAL AND ITS RECEIPT (AS PASTED)

```
TASK [Admission control - refuse before the clone spends anything] ***
fatal: [localhost]: FAILED! => {
"assertion": "ip_collision | length == 0",
"evaluated_to": false,
"msg": "Admission refused (nothing was created): 192.168.122.60 -
already assigned to VMID 103 (vm-faz4-01); pick another
IP or retire that VM first"
}

PLAY RECAP *********************************************************
localhost : ok=14 changed=0 failed=1

# qm list after the run: 100 alpine, 101 alpine-b, 103 vm-faz4-01,
# 9000 template. vm-collide-test does not exist. Zero footprint.
```

One subtlety worth keeping: in this fresh request the budget sums task counted vm-faz4-01, iterating it with ok rather than skipping, because target_vmid is 0 when the hostname does not exist yet. The target exclusion only applies to adoption re-runs; a fresh request counts every VM in the fleet, which is exactly what a gate should do. The red task stayed in Semaphore's history on purpose; refusals are audit records, not clutter, and deleting them would be deleting the evidence that the gate works.

Test C: the squatter, once by accident

Test C's first attempt misfired in the most human way possible: the operator re-used the previous form and forgot to change the IP field, so the request went to .60 again and was refused as a collision, correctly, for the second time. The misfire turned out to be worth more than a clean run would have been, because that request satisfied two failure conditions at once, an IP collision and a probe that answers (vm-faz4-01 is alive), and the assert reported the collision as THE failed assertion. The priority routing in

the fail message, most actionable first, was now field-proven: when both branches could fire, the one that names a VMID wins.

The re-run went to 192.168.122.10, sem-01's own address, chosen because it is the one IP in the lab guaranteed to answer TCP/22 while no VM holds it in ipconfig0. Every prediction matched, five of five: the probe task green because SSH answered; the failed assertion exactly the probe condition; evaluated_to false; the squatter message to the character; and the recap reading failed=1 with no ignore, because a probe that answers does not time out. The last detail is the fingerprint of the branch: Test D's probe against an empty address would read ignored=1 instead.

TEST C, 09:12, THE SQUATTER REFUSAL (AS PASTED)

```
TASK [Probe the requested IP (anything listening on SSH?)] ******
ok: [localhost] => {"changed": false, "elapsed": 0, "port": 22}

TASK [Admission control - refuse before the clone spends anything] ***
fatal: [localhost]: FAILED! => {
"assertion": "ip_probe.failed | default(false) | bool or ip_owned_by_target | bool",
"evaluated_to": false,
"msg": "Admission refused (nothing was created): 192.168.122.10 -
something is ALIVE there (TCP/22 answered) but no VM holds
it in ipconfig0 - a manual or DHCP machine squats that
address; pick another IP"
}

PLAY RECAP *********************************************************
localhost : ok=14 changed=0 failed=1 ignored=0
```

Test D: the budget, exact to the character

Test D requested 2 vCPU and 16 GB at a free address, against a fleet then holding 5 vCPU and 7 GB. Both numbers matter: the request passes every enum and every per-request cap, so the only thing that can refuse it is the aggregate, and only the RAM line can trigger, since 7 vCPU plus 2 fits inside 8. The refusal printed the arithmetic with the fleet it would create, 7/8 vCPU and 23/16 GB, and pointed at the knobs file by name. The probe timed out at the empty address exactly as a fresh-IP probe should, which is why the recap carries ignored=1.

TEST D, 09:05, THE BUDGET REFUSAL (AS PASTED)

```
TASK [Probe the requested IP (anything listening on SSH?)] ******
fatal: [localhost]: FAILED! => {"changed": false, "elapsed": 3,
"msg": "Timeout when waiting for 192.168.122.99:22"}
...ignoring

TASK [Admission control - refuse before the clone spends anything] ***
fatal: [localhost]: FAILED! => {
"assertion": "allocated_ram_mb | int + spec_ram_gb | int * 1024
<= capacity_max_ram_gb | int * 1024",
"evaluated_to": false,
"msg": "Admission refused (nothing was created): 192.168.122.99 -
budget exceeded: adding 2 vCPU / 16 GB would take the
fleet to 7/8 vCPU and 23/16 GB allocated; lower the
request or decommission a VM (see
vars/lab-environment.yml capacity_max_*)"
}

PLAY RECAP *********************************************************
localhost : ok=14 changed=0 failed=1 ignored=1
```

Closing the gate, 4 of 4

Test	When	Branch	The evidence that closed it
A, admission grant	run #41	the gate opens	success message with live math 5/8 vCPU, 7/16 GB; receipt carries the fleet line; recap ok=21 skipped=2 (the 5a pin); the alpine correction proved the gate reads live state, not history.
B, collision	08:12	ipconfig0 owner	refusal names VMID 103 (vm-faz4-01); failed assertion ip_collision | length == 0 evaluated_to false; ok=14 changed=0 failed=1; qm list clean.
D, budget	09:05	aggregate capacity	refusal prints 7/8 vCPU and 23/16 GB, the fleet the request would create; RAM inequality the trigger; ignored=1 from the fresh-IP probe timeout.
C, squatter	09:12	TCP/22 liveness	probe answers at the runner's own IP; the probe condition fails with the squatter message; ignored=0 because the probe answered. The 09:04 misfire additionally proved the collision branch outranks the squatter when both apply.

Table 4. The admission gate's field scoreboard. Four branches, four proofs, all inside seventy minutes on 30 August 2026, every refusal before the clone with zero qm footprint.

Phase 5a closed on that table, and the manner of its closing is worth as much as the code. Every refusal was red in Semaphore by design and left in place as the audit trail. Every refusal produced changed=0, which is the difference between a gate and a leak: a gate that created something before refusing would need a cleanup playbook, and this one never will. And the one prediction that missed, the alpine fleet,

missed in the direction that proves the design: the gate was right because it read the cluster, and the prediction was wrong because it read the git log. The next phase would lean on the same principle, from the other direction: remembering what live state cannot.

4. The Watchman: 08_health_audit.yml

The morning/evening ask is the oldest one in the chat: a Proxmox health check twice a day, with mail. The shipped answer is a standalone playbook that is never chained into the deploy path, runs read-only against the cluster and the guests, and ends by rendering a digest, mailing it if the runner is armed, refreshing a baseline, and only then deciding whether the task should be red. Semaphore schedules it; nothing else touches it.

Two design rejections, both at source level

The roadmap's wording for 5b was a drift audit with an intent registry, and the first design idea that died was the most natural one: store each VM's intent in its own notes field, right where the VM lives. It died at the source, not in review: community.proxmox 2.0.0's proxmox_kvm has no notes parameter at all, verified in the module source before any code was written. The second idea, a registry file the deploy writes, died the same way the 5a registry did, it would be a second thing that can drift. What survived is the inversion: the audit keeps its own baseline, a full snapshot of the managed fleet written at the end of every run, and drift is the difference between live state and the last audit.

That inversion carries the one capability nothing else in the stack has. Live state cannot remember a VM that no longer exists; the baseline can. A deleted managed VM is the drift class that justifies the whole mechanism, because after an out-of-pipeline deletion every other source of truth, the ip_records the gate reads, the monitoring, the inventory, is quietly lying about reality. And the fleet membership question needed no registry either: play 00 has been stamping every deployed VM with the pipeline-managed tag since Phase 4, so membership already lives on the cluster. The audit reads the tag from live config; a manual qm set that adds or removes the tag shows up as drift, not as an error, which is the correct severity for a fact.

Three plays, one verdict

The playbook is three plays. Play 1 is the API view from localhost: cluster status (quorum, and a standalone node is normal, not a finding beyond a note), node membership, node health (status, pressure thresholds, recent boots), the fleet budget summed exactly as the admission gate sums it, the infrastructure CTs that must be running, the golden template that must exist and stay stopped, the managed fleet built from the tag, drift against the baseline, per-VM API checks (stopped, onboot, agent, ipconfig0), and a TCP/22 probe per running managed VM to decide who the guest layer can reach. Play 2

is the guest view over SSH: read-only, ignore_unreachable, three service states via systemctl is-active, filesystem usage from df -P, findings folded into a per-host report that play 3 reads back as host facts. An unreachable guest becomes a finding, never a dead audit.

Play 3 is where the design earns its keep. It collects the guest reports, cross-checks probes that answered against reports that never arrived (an auth failure is a finding of its own), normalizes empty-safe defaults, renders the digest from the template, prints it to the task log, mails it if and only if a transport exists and mail is configured, rewrites the baseline, and asserts the verdict LAST. The ordering is the whole trick: the log cannot fail, the mailbox can, so the digest is printed before anything fallible runs, and a red task is guaranteed to mean the digest is already readable above the fold.

Severity	Means	Members
CRITICAL	a human should read the digest now	quorum lost, a node down, an infrastructure CT down, a managed VM deleted since the last audit.
WARN	a real problem that is not an emergency	managed VM stopped, onboot=0, probe failure, SSH auth failure, a service down, a filesystem at or past 80%, node memory at or past 85% or CPU at or past 90%, budget exceeded by manual change (someone qm set past the gate).
INFO	a fact, not an alarm	drift lines (cores or IP changed), a new managed VM, a standalone node note, template issues, mail not configured.

Table 5. The severity model, verbatim in spirit from the playbook header. The verdict assert fails on CRITICAL and WARN only; pure-INFO runs end green, which is why a green task is meaningful and a red one is urgent.

The digest

The digest is one Jinja template, audit-mail.j2, and its shape is the product spec: it opens with a summary block, then cluster, nodes, fleet, managed fleet, guests, and findings, and every section has an else branch because the interesting runs are exactly the ones where a section is empty. The subject line carries the verdict, so the inbox list alone triages:

TEMPLATES/AUDIT-MAIL.J2, THE HEADER AND THE FINDINGS BLOCK (VERBATIM)

```
To: {{ audit_mail_to }}
From: {{ audit_mail_from }}
Subject: [proxmox-audit] {{ audit_summary.worst }} -
{{ audit_summary.vm_total }} VM(s), {{ audit_summary.ct_total }}
CT(s) - {{ audit_summary.when }}

... cluster, nodes, fleet, managed fleet, guests sections ...

FINDINGS
{% for f in audit_findings %}
[{{ f.sev }}] {{ f.text }}
{% else %}
(none - a quiet fleet)
{% endfor %}

Baseline: {{ audit_baseline_file }} (refreshed by this run - the
next audit diffs against it)
```

Two branches of that template earned field proof before the phase closed: the findings loop with content (three reboot INFO lines in the first green run) and the else branch, (none - a quiet fleet), which the first scheduler-driven digest rendered when the fleet had nothing to say. Both matter; a template that only renders the happy path was never tested until it failed.

The knobs, and what HOME has to do with them

VARS/LAB-ENVIRONMENT.YML, THE AUDIT SECTION (KEY LINES)

```
audit_baseline_dir: "{{ lookup('env', 'HOME') | default('/root', true)
}}/proxmox-audit-baseline"
audit_fs_warn_pct: 80 # guest filesystem usage that deserves a WARN
audit_node_mem_warn_pct: 85 # node memory pressure threshold
audit_node_cpu_warn_pct: 90 # node CPU pressure threshold
audit_guest_services: # the services every managed VM owes the fleet
- zabbix-agent2
- rsyslog
- qemu-guest-agent
audit_mail_enabled: true
audit_mail_from: "hakanismail53@gmail.com" # must match the msmtp account
audit_mail_to: "" # RETIRED 30 Aug 2026: the Gmail app password was revoked
# after the full mail chain was field-proven; empty =
# digest stays in the task log (graceful log-only mode).
# To re-arm: new app password via scripts/sem-01-msmtp.sh
# + set this back to the recipient address.
```

The baseline directory line looks innocuous and carries the biggest lesson of the 5b field days: it resolves under whatever HOME the task process carries. On sem-01 the Semaphore service user's home is /home/semaphore, not /var/lib/semaphore where the project's own early runbook assumed it would be, and the first green audit proved it by printing the real path in its own digest. The file's comment block now records the field-proven path and the consequence that matters: run the scheduled audit from one

place, because two runners means two baselines drifting apart.

The sandbox harness, again, and its catch

Like the gate, the audit went to the field with a simulation harness: eighteen scenarios, expressions copied verbatim, module reads replaced by fixtures shaped per the source, and pins on the findings that matter (a steady-state run produces zero findings; a budget manually pushed to 9/8 produces the WARN; a deleted VM produces the vanished line). Eighteen of eighteen passed, and the harness caught one real playbook bug en route: the new-VM detection used | keys() as if it were a Jinja filter, which it is not, and both the sim and the playbook had the same wrong expression, the dict2items form fixed both. That is the second verbatim-expression bug a pre-field harness has caught in this series, and the reason the pattern is now house style: the sim does not prove the playbook correct, it proves the expressions parse and the branch logic routes, which is exactly the class of bug that otherwise costs a field round.

5. The Watchman’s First Field Steps

The audit’s first four runs are a story about a runner, not a playbook. Three of them died red, each one layer deeper than the last, and none of the three was a bug in 08_health_audit.yml. The playbook had been sandbox-proven; the machine it ran on was the variable. What follows is the onion, peeled one layer per run, with the errors exactly as the task logs carried them.

Layer one: the collection that was never installed (#48)

The first Health Audit run, task #48, died at the playbook’s first module task:

TASK #48, 30 AUGUST, THE FIRST MODULE TASK

```
TASK [Read the cluster status (quorum and node membership)] ***
fatal: [localhost]: FAILED! => {
"msg": "couldn't resolve module/action 'community.proxmox.proxmox_cluster_status_info'
in the search path ..."
}
# died at 08_health_audit.yml:64, before any state change; the
# audit is read-only, so there was nothing to clean up.
```

The archaeology took one paste. The runner’s collection list showed community.proxmox 1.4.0 sitting beside community.mysql 4.0.1, community.okd 5.0.0, and community.postgresql 4.2.0, which is the fingerprint of Ubuntu’s ansible deb bundle, not a galaxy install. The setup script’s own galaxy step had never landed on this machine, silently, months of runs ago; the deb’s 1.4.0 had been serving everything ever since. Deploy VM never noticed because proxmox_kvm and proxmox_vm_info exist in every version ever shipped; 08 was the first playbook in the repo to use the newer cluster and node info

modules, and it asked the question the runner could not answer.

Layer two: the HOME the task never reads (#50)

The first fix, installing community.proxmox:2.0.0 as the service user with HOME pointed at /var/lib/semaphore, verified cleanly under ansible-doc, and the re-run, task #50, failed with the identical error. An install that is provably present and a task that cannot see it has exactly one class of explanation: the task process's environment is not the interactive one. The diagnostic was one command, reading the service's actual environment out of /proc:

THE ENVIRON DIAGNOSTIC, AND WHAT IT SHOWED

```
$ cat /proc/$(systemctl show -p MainPID --value semaphore)/environ \
| tr '\0' '\n' | grep -E '^HOME'
HOME=/home/semaphore

# The service user's home is /home/semaphore (useradd -m in
# setup step 2), so the task's ~/.ansible is
# /home/semaphore/.ansible, and the install that went to
# /var/lib/semaphore/.ansible is invisible to every task.
# The fix: install to the HOME-independent system path.
$ sudo ansible-galaxy collection install community.proxmox:2.0.0 \
-p /usr/share/ansible/collections --force
```

The system-path install is the hypothesis-independent answer: Ansible's search order passes through /usr/share/ansible/collections in every context, HOME or no HOME, so the collection resolves for the interactive shell, the task process, and any future runner alike. The setup script was hardened in the same window, step 8 now installs the pinned 2.0.0 as root into that path with the whole story in a comment block, so a rebuilt sem-01 cannot grow this onion again.

Layer three: the Python floor (#51)

Task #51 got one module deeper: cluster status resolved and ran green, and the node status task died on a dependency the collection declares but the machine did not meet:

TASK #51, ONE LAYER DEEPER

```
TASK [Read the node status (load, memory, uptime - the /nodes view)]
fatal: [localhost]: FAILED! => {
"msg": "Requires proxmoxer 2.3 or newer; found version 2.2.0"
}

$ sudo pip3 install --break-system-packages 'proxmoxer==2.3.0'
$ python3 -c "import proxmoxer; print(proxmoxer.__version__)"
2.3.0
```

The floor is a Python library, not a collection: Ubuntu 26.04's python3-proxmoxer apt package carries 2.2.0, and community.proxmox 2.0.0's newer info modules want 2.3. The pip copy lands in /usr/local and

shadows the apt one; it is pure Python, no daemon, no restart. Why did three dozen green deploys never hit it? Because proxmox_kvm and proxmox_vm_info, the deploy path's two modules, do not declare the floor. And why did the sandbox never hit it? Because the sim harness feeds the playbook fixtures, so no module ever imported the library at all. Both are honest limits of the harness pattern, and both are now covered: the setup script pins proxmoxer==2.3.0 in step 1, and the sim's job description says expressions and routing, not dependencies.

First green (#52), and the doc bug it found

Task #52 was the first fully green audit: localhost ok=27 changed=4 failed=0, vm-faz4-01 ok=5 changed=0, the guest layer provably read-only on its first contact. The verdict printed the designed first-run shape:

TASK #52, THE VERDICT AND THE DIGEST'S OPENING (CONDENSED)

```
TASK [Verdict - red means "a human should read the digest"] ****
ok: [localhost] => {
"msg": "AUDIT OK: 5 info note(s), digest printed above.
Baseline refreshed for the next run."
}

PROXMOX HEALTH AUDIT - 2026-08-30 ...
Verdict: OK (0 critical / 0 warn / 5 info)
CLUSTER quorum: yes nodes: pve-a, pve-b, pve-c
NODES pve-a cpu=2% mem=54% up=0h ... (all under thresholds)
FLEET allocated: 5/8 vCPU, 7/16 GB (templates and CTs excluded)
MANAGED vm-faz4-01 VMID 103 running 2 vCPU / 4 GB ip=192.168.122.60 disks=2
GUESTS vm-faz4-01 up 0h services: all active
FINDINGS [INFO] pve-a node rebooted recently (uptime ...) ...
[INFO] vm-faz4-01 (VMID 103) is new since the last audit -
baselined now
[INFO] mail not configured on the runner - digest printed
to this task log only
Baseline: /home/semaphore/proxmox-audit-baseline/fleet.yml
```

Five INFO lines, each a design promise kept: the reboot detector fired on all three nodes because the lab had been rebooted half an hour earlier (the threshold is one hour of uptime); the new-VM line fired because there was no baseline yet and the first run baselines everything; the mail line fired because mail was not configured yet, which is exactly what the digest should say when the fallback is the log. And the last line found a documentation bug of ours: the baseline path is /home/semaphore, not the /var/lib/semaphore the runbook instructions had assumed, which retroactively explained the whole layer-two mystery and got corrected in the vars file the same day. The digest also signs unknown@sem-01, because the task environment carries no USER variable; known, cosmetic, kept.

Drift-zero (#53), the proof the design wanted

The second run was the design's real examination: with a baseline on disk, does an unchanged fleet produce a clean diff? Task #53 answered yes with every prediction hit: the baseline load ran instead of skipping, the drift report iterated vm-faz4-01 and produced zero findings, the deleted-VM and new-VM tasks both skipped, the new-VM INFO was gone (five INFO down to four), and the verdict stayed green. The bonus proof nobody had predicted: the baseline rewrite task reported ok, not changed, because the copy module saw byte-identical content. An idempotent file rewrite is a quiet thing to prove, and it is the difference between a baseline that churns and one that only moves when the fleet does.

RUN #52 VS RUN #53, THE RECAPS (AS PASTED)

```
# first run (baseline created):
localhost : ok=27 changed=4 failed=0
vm-faz4-01 : ok=5 changed=0 failed=0

# second run (baseline read, drift-zero, byte-identical rewrite):
localhost : ok=27 changed=2 failed=0
vm-faz4-01 : ok=5 changed=0 failed=0

# changed=4 -> changed=2 decomposes exactly: baseline dir +
# baseline write converge to ok; what stays changed is the
# register-guests add_host and the digest render, by design.
```

The confidence re-run (#55), and two benign anomalies

The collection and library upgrades sit under the deploy path too, so one adoption re-run closed the loop: task #55 went green with the exact profile of the pre-upgrade run, localhost ok=21 changed=2 skipped=2, vm-faz4-01 ok=86, admission line 5/8 vCPU and 7/16 GB, receipt printed, and the old INJECT_FACTS_AS_VARS deprecation warning that had been riding along since Phase 4 gone from the delivery task, hard proof the ansible_facts fix from the same push was live. Two anomalies rode with it, both benign, both worth a paragraph in a guide that promises honesty.

The first: the pull phase opened with repeated "error: object file .git/objects/8e/3f8bec... is empty" and "fatal: fetch-pack: invalid index-pack output", then Semaphore fell back to a fresh clone and the run finished green. A fetch interrupted mid-write leaves a truncated object in the template's cached clone; Semaphore notices the unusable cache and re-clones. The run is unaffected, nothing to clean by hand, and the README's troubleshooting table now carries the row so the next person seeing it does not reach for a backup. The second: vm-faz4-01 reported an uptime of 2.7 minutes at a moment when no playbook had touched it; the operator had rebooted it out-of-band minutes before the run. The pipeline adopted the rebooted VM cleanly, an accidental bonus proof that the whole chain survives an out-of-band reboot, and the audit's reboot detector, which had fired on the nodes an hour earlier, would have caught the guest side too were the uptime threshold per-guest.

With #52 and #53 green and the deploy path re-proven, the core of 5b was field-closed: the baseline exists, drift against it is zero when nothing changed, and every severity class above INFO remained

simulation-proven and waiting for the field to hand them real events. The field obliged within hours, in chapters 6 and 7, with a mailbox, a queue, and a deleted VM.

6. The Mail Path and the Schedule

The ask said mail, and mail is two problems wearing one word: the transport, which lives on the runner and involves a password nobody wants in git, and the schedule, which turns the watchman from a button into a habit. Both closed on 30 August 2026, and the closing sequence produced the single most quoted artifact of the phase: a digest in a Gmail inbox whose subject line had been predicted to the character.

msmtp on the runner, once

The transport is one apt package and one config file. scripts/sem-01-msmtp.sh is the one-time setup, run as root on sem-01: it installs msmtp (no daemon, it is a send-only client), writes /etc/msmtprc pointing at smtp.gmail.com:587 over STARTTLS with a Gmail app password, sets the file to mode 600, and then writes a second copy to the service user's home via getent, because the audit task runs msmtp as the semaphore user and msmtp prefers the per-user config, which dovetails with the HOME finding of chapter 5. The app password never touches the repository; the envelope addresses (audit_mail_from, audit_mail_to) travel in lab-environment.yml, but the secret lives only in the config files on the runner.

The script's final hardening block exists because of a trap the field never got to spring, found by reading msmtp's behavior instead: the root test send leaves /var/log/msmtp.log owned by root, msmtp treats an unopenable logfile as fatal, and the audit runs it as the semaphore user. Without the fix, the first mailed audit would have gone red at the mail step with the digest already safely in the task log. The script now pre-creates the logfile with the right ownership before the root test send, and the README's mail row carries the chown one-liner for anyone who hand-rolls the setup.

On the playbook side, sending is gated three ways: the runner must have msmtp, audit_mail_enabled must be true, and audit_mail_to must be non-empty. Any leg missing and the digest stays in the task log with an INFO line saying so, which is the graceful degradation the whole design leans on. One topology correction from this window survives into the repo docs because the operator hit it in practice: the script runs on sem-01, reached with pct enter 901, and the bare repo lives on ctrl-01 at .10; an edit made to a checkout on sem-01 is an edit to a file no task ever reads, a distinction that cost one wasted nano session and is now written down.

The mail-proof run (#56), subject line exact

With the transport proven by the script's test mail and the envelope pushed through the repo, one manual audit run closed the whole chain:

THE MAIL-PROOF DIGEST, AS IT LANDED (SUBJECT AND VERDICT)

```
Subject: [proxmox-audit] OK - 4 VM(s), 2 CT(s) - 2026-08-30 08:56 UTC
(delivered to the inbox at 11:56 Istanbul; task -> msmtp as the
semaphore user -> smtp.gmail.com:587 -> inbox)

TASK [Mail finding] ********************************************
skipping: [localhost]

TASK [Mail the digest] *****************************************
ok: [localhost]

TASK [Verdict - red means "a human should read the digest"] ****
ok: [localhost] => {
"msg": "AUDIT OK: 3 info note(s), digest mailed to
hakanismail53@gmail.com."
}

PLAY RECAP *****************************************************
localhost : ok=27 changed=2 failed=0
vm-faz4-01 : ok=5 changed=0 failed=0
```

A mail in an inbox proves more than a mail in an inbox. It proves the push happened (the task clones the bare repo, and the non-empty audit_mail_to only exists in the new commit); it proves the three-part gate passed and the send ran as the service user without dying on the logfile; it proves the Jinja-rendered headers parsed as an RFC 822 envelope that msmtp -t accepted; and it proves the subject line's conditional verdict rendering, because the OK in it was computed by the playbook, not typed by anyone. The prediction scoreboard for that run closed with the caveat branch hit rather than the primary: three reboot INFO lines, because the lab had rebooted again an hour before, which the uptime detector reported faithfully. A detector that fires on real reboots three times in one morning is not flaky; it is employed.

The scheduler, proven early by impatience

The documented plan was to schedule the audit at UTC 04:00 and 16:00, Turkish 07:00 and 19:00, the server's clock being UTC. The operator did something better for the proof and worse for the mailbox: a test schedule at every five minutes, to watch a non-manual fire the same morning. It fired, the digest arrived at 09:10 UTC, and it was the quietest the fleet would ever be:

THE FIRST SCHEDULER-DRIVEN DIGEST, 09:10 UTC

```
Subject: [proxmox-audit] OK - 4 VM(s), 2 CT(s) - 2026-08-30 09:10 UTC

Verdict: OK (0 critical / 0 warn / 0 info)

FINDINGS
(none - a quiet fleet)

# the reboot INFOs had aged out (nodes past 1h uptime), the
# new-VM line was baselined, mail was armed: zero findings,
# the empty-findings branch rendered for the first time.
```

The instruction that followed was the most urgent sentence of the phase: delete the five-minute schedule now. A read-only audit every five minutes is harmless to the lab and catastrophic to an inbox, roughly 288 digests a day, plus a task history nobody could read anymore. The production replacement is a single cron expression, 0 4,16 * * *, and the schedule's timezone arithmetic is worth stating precisely because it is the kind of thing that fires at the wrong hour for a week before anyone notices: sem-01 runs UTC, the operator runs Istanbul at UTC+3, so 07:00 and 19:00 local are 04:00 and 16:00 UTC. Check date on the runner before trusting the cron field.

Retirement, and the order that got inverted

Every app password has a life cycle, and the project had written the correct one down: first push audit_mail_to: "" so audits degrade gracefully to log-only, then revoke the password at Google. The operator, tidying up, did it the other way round: revoked first, and the next audit, task #63, went red at the mail step:

TASK #63, THE RETIRED PASSWORD MEETS THE MAIL TASK

```
TASK [Mail the digest] *********************************************
fatal: [localhost]: FAILED! => {
"changed": false,
"rc": 77,
"stderr": "msmtp: authentication failed (method PLAIN)\n 535-5.7.8 Username and
Password not accepted.
... BadCredentials"
}

# the digest itself is in the task log above this task,
# by design; the baseline write and verdict never ran.
```

The inverted order accidentally field-proved the branch the graceful path can never show: a configured-but-broken mail path is FATAL on purpose. An alerting channel that fails silently is worse than no channel, because it manufactures confidence; a red task with the digest in its log is exactly the loudness the design wants. No digest was lost, and the sequencing detail is instructive: the mail task runs before the baseline write and the verdict, so #63 never refreshed the baseline, harmless here because the fleet had not changed and the next run's baseline would have been byte-identical anyway. The retirement

was then completed the documented way, audit_mail_to back to empty, and the audit returned to log-only mode with its INFO line, with audit_mail_from kept so that re-arming is one new app password and one two-line edit.

Mail state	What the audit does	Task outcome
No msmtp on the runner	INFO: mail not configured, digest printed to the task log only; send skipped	green
Transport armed, recipient set	digest printed, then mailed as the service user; verdict says mailed to whom	green (or red if findings warrant, with the digest in both places)
Transport armed but broken (bad password, dead relay)	digest printed, then the send fails; four downstream tasks die with it	red at the mail step, by design: a broken alert channel must be loud
Retired (audit_mail_to empty)	INFO: mail not configured, digest stays in the log; send skipped	green; re-arming needs a new app password plus the one-line edit

Table 6. The mail gate's four states, all four of them field-observed on 30 August 2026, two of them by plan and two by accident.

7. The Accident: Concurrency (5c)

The roadmap called it a parallelism stress test, and the design write-up was honest about what it feared: the admission gate is check-then-act with no lock. Task one reads live state, task two asserts on it, task three clones. Under true parallelism, two requests can both pass the assert before either lands its clone, and the fleet ends up over budget or, worse, two VMs holding the same IP in ipconfig0. The field then declined to cooperate with the fear, in the most informative way possible.

The fix candidates, parked until evidence

The obvious fixes all have teeth. A flock around the play cannot span Ansible module tasks cleanly; a claim file needs garbage collection the moment a run dies mid-flight; a database-level lock is a registry wearing a uniform, and the registry idea was already dead. The candidate that survived the design review is post-apply re-verification: after the clone and spec land, re-read the ipconfig0 records and the budget; if another VM with a lower VMID now holds my IP, destroy my fresh clone and refuse cleanly, lowest VMID wins, the same for the budget with the highest VMID rolling back. Stateless, nothing to garbage collect, and consistent with the API-is-the-truth philosophy. But it was deliberately not built, because all of it is only necessary if the race is reachable, and reachability was an empirical question about

Semaphore, not about the playbook.

Round 0: the queue answers first (#58, #59)

The experiment was two browser tabs, two valid distinct requests submitted within a second of each other: vm-c-01 at .61 and vm-c-02 at .62, both minimal specs. If Semaphore ran them concurrently, the predictions said, both would read the fleet at 5/8 vCPU, both would pass, and the fleet would land at 9/8: the budget violation, live. What happened instead: task #58 ran the full pipeline, a fresh clone with VMID 104, every play green to the receipt, localhost ok=21 changed=4, vm-c-01 ok=69 changed=13, the first full fresh-clone run of the 5a era. Task #59 did not run at all until #58 finished. Then it ran, read the fleet as it now stood, and was refused:

TASK #59, THE REFUSAL THAT PROVED THE QUEUE (AS PASTED)

```
TASK [Sum allocated vCPU and RAM (templates and the target excluded)]
ok: [localhost] => (item=alpine)
ok: [localhost] => (item=alpine-b)
ok: [localhost] => (item=alpine-c)
ok: [localhost] => (item=vm-faz4-01)
ok: [localhost] => (item=vm-c-01) <-- the task AHEAD of us

TASK [Admission control - refuse before the clone spends anything]
fatal: [localhost]: FAILED! => {
"msg": "Admission refused (nothing was created): 192.168.122.62 -
budget exceeded: adding 2 vCPU / 4 GB would take the
fleet to 9/8 vCPU and 15/16 GB allocated; lower the
request or decommission a VM ..."
}

# the sums iterate vm-c-01, created seconds earlier by the task
# ahead of us in the queue: the second task read the first
# task's COMMITTED state.
```

Three independent proofs sat inside #59's log that it ran after #58's critical section: the repo prep printed "Already up to date" because #58 had already pulled; the IP records task collected vm-c-01 with its ipconfig0, which only exists after the apply; and the budget sum included vm-c-01. The predictions that mattered were exact, 7/8 vCPU and 11/16 GB granted for the first request, 9/8 and 15/16 refused for the second, and the parallel-branch predictions simply never got to happen. The operator's own reading of the result is the field finding, verbatim in spirit: the error was expected, what interested him was the waiting. The queue is the mutex. Nobody built an admission lock; the runner serialized its way into providing one.

The cross-template probe (#60, #61)

Round 0 proved same-template serialization. But the queue's scope was still open: if two different templates run concurrently, a deploy and an audit, say, the race would be reachable again, this time between the audit's read and the deploy's write. The probe was one adoption re-run of vm-faz4-01, budget-safe, idempotent, several minutes of runtime, with a Health Audit fired from a second tab about a

minute in. Task #60 went green with the familiar profile, ok=21 changed=2 skipped=2 on localhost, vm-faz4-01 ok=86 changed=1, and one lovely detail: the admission probe returned ok rather than timing out, because .60 answers, it is vm-faz4-01 itself, the adoption branch of the gate exercised live. Task #61, the audit, ran green too, and carried the first two-guest digest, vm-faz4-01 and the morning's vm-c-01:

TASK #61, THE AUDIT THAT WAITED (KEY LINES, AS PASTED)

```
TASK [Drift report - live state vs the last audit] *************
ok: [localhost] => (item=vm-faz4-01)

TASK [New managed VMs (first audit - the baseline starts here)]
ok: [localhost] => {
"msg": "[INFO] vm-c-01 (VMID 104) is new since the last audit -
baselined now"
}

Subject: [proxmox-audit] OK - 5 VM(s), 2 CT(s) - 2026-08-30 10:23 UTC
PLAY RECAP: localhost ok=28 changed=3 vm-faz4-01 ok=5 vm-c-01 ok=5
```

The timing is the finding. Reconstructed from timestamps inside both logs: the deploy's tail, the security agents and the delivery gate, ends around 10:20 to 10:21, and the audit's guest-layer service checks carry 10:22:59 stamps, so the audit began play 1 only after the deploy finished, having been fired, per the operator's own account, during the deploy's middle. It sat four to five minutes in the queue. The honest caveat stays attached, because the one-word confirmation from the Tasks view never arrived: the reconstruction rests on task-internal timestamps, and the alternative reading, that the audit was simply fired late, cannot be excluded from the logs alone. The README's roadmap row cites it with exactly that caveat, and the conclusion is stated at the strength the evidence supports: the queue is global across templates, per the timestamps, with no parallelism knob anywhere in the UI or the docs.

What the queue means

The concurrency conclusion writes itself from the two rounds together. The admission gate's check-then-act window is unreachable from the Semaphore UI, same-template and cross-template alike, because the queue serializes everything. The remaining race vectors are the CLI and the API, where two ansible-playbook processes genuinely can overlap, so the post-apply re-verification design stays written down in the project memory as defense-in-depth for the day this pipeline grows a second front door. And the queue has an operational cost that is now a documented production note: a long deploy queues everything behind it, other requests and scheduled audits alike, so the evening digest waits out the evening deploy. Acceptable in a lab; in production, plan maintenance windows.

Round 3: the alarm (#62), and a prediction miss kept

The cleanup round doubled as the field proof the deleted-VM detector had been waiting for since the design table was written. vm-c-01 had served its purpose; the destroy came first, with one operational lesson en route, qm destroy refused to purge a running VM, so qm stop first, then destroy. The audit that

followed, task #62, found the deletion, mailed it, and went red, exactly as the design intended and exactly NOT as predicted:

TASK #62, THE DELETED-VM ALARM (AS PASTED, CONDENSED)

```
TASK [Deleted managed VMs (only the baseline can remember these)]
ok: [localhost] => {
"msg": "[CRITICAL] managed VM vm-c-01 (VMID 104) vanished
since the last audit"
}

Subject: [proxmox-audit] CRITICAL - 4 VM(s), 2 CT(s) -
2026-08-30 10:28 UTC

PLAY RECAP *********************************************************
localhost : ok=27 changed=3 failed=1

# exit status 2: the task is RED on purpose, so the alarm is
# visible in the task list without anyone opening the mail.
```

The prediction had said an INFO line and a green task; the field said CRITICAL and red, and the miss is kept on this page because of why the field was right. The severity table in chapter 4 is the shipped code, and it has always listed a deleted managed VM under CRITICAL; the prediction misremembered the table it was quoting. The design reasoning is worth restating, because it is the strongest sentence in the audit: a new VM is curiosity, a vanished VM is an alarm, because after an out-of-pipeline deletion every pipeline artifact, the gate's IP records, the baseline, the monitoring, is quietly lying about reality. The same run carried the supporting numbers: the fleet back to 5/8 vCPU and 7/16 GB, the seventh independent confirmation of that arithmetic, and node memory on pve-a dropping from 84 to 41 percent, the morning's pressure watch resolved by exactly the VM it was watching.

The self-heal (#63)

The baseline written at the end of the red run had already dropped VMID 104, so the next audit was predicted quiet, and the confirmation run, task #63, delivered: verdict OK with zero critical, zero warn, zero info, the findings section rendering its else branch, (none - a quiet fleet), the vanished line gone without anyone doing anything. The mail step of that same run then failed on the retired app password, which is the chapter 6 story and cost nothing but the red task: the digest was in the log above it. The recap arithmetic of #63 is the last decomposition of the phase, and every number in it is accounted for: ok=22 skipped=9 failed=1 against #62's ok=27 skipped=8, the deleted-VM task moving from one-ok-one-skip to zero-ok-two-skip, and the mail failure killing the four downstream tasks, snapshot, directory, baseline write, verdict, that would have followed it.

Round	Tasks	What it proved
Round 0, same template	#58, #59	two back-to-back Deploy VM submissions do not race; the second waits, then reads the first's committed state and is refused at 9/8 vCPU and 15/16 GB. The queue is an accidental admission mutex.
Cross-template probe	#60, #61	an audit fired mid-deploy started only after the deploy's tail, per task-internal timestamps: the queue is global, not per-template. The caveat is stated with the finding.
Round 3, the alarm	#62	out-of-pipeline qm destroy of a managed VM produces CRITICAL, a red task, and a CRITICAL-subject mail. Only the baseline can remember the deletion; live state cannot.
Self-heal	#63	the next audit is quiet, 0/0/0, the vanished line gone via baseline refresh; the mail step's death on the retired password proved the fatal-by-design branch on the way out.

Table 7. The 5c evidence chain, four rounds, no new code: everything the stress test set out to learn was learned by watching the queue.

8. Troubleshooting

The README's troubleshooting table grew thirteen Phase 5 rows across the two field days, and the shape of them tells the phase's story better than any summary: three refusals that are the product working, two runner layers that were never the playbook's fault, four mail states, two red-by-design outcomes, one lesson about pushing before proving, and one anomaly that fixes itself. The rows below are the field-tested core of that set.

Symptom	Cause	Fix
Deploy dies in 00 at "Admission control": already assigned to VMID 103 (vm-faz4-01)	another VM's ipconfig0 already holds the requested IP (the 5a collision gate)	pick another IP or retire that VM; the refusal happened before the clone, so nothing was created and a re-run with the corrected form is safe
Deploy dies in 00 at "Admission control": something is ALIVE there (TCP/22 answered)	no VM holds the IP in ipconfig0 but a machine answers on port 22: a manual VM, a DHCP box, an LXC, anything with sshd (the liveness probe; ipconfig0 cannot see these)	pick another IP; if the squatter is stale, retire it first
Deploy dies in 00 at "Admission control": budget exceeded	adding the request would push allocated non-template qemu vCPU/RAM past capacity_max_* (templates and CTs are infrastructure, excluded by design)	lower the request, decommission a VM, or raise the budget in vars/lab-environment.yml after checking the node's real headroom
A run you expected to carry new behavior came back green with the old numbers (run #38 shape)	the new code was never pushed: Semaphore tasks clone the bare repo on ctrl-01, so uncommitted or unpushed edits are indistinguishable from no edits	read the pull line's hash range in the task log; extract, commit, push on ctrl-01, verify the new HEAD, re-run. The recap arithmetic (ok=16 vs ok=21 on the adoption path) is the second signature
Health Audit dies at its very first task: couldn't resolve module com munity.proxmox.proxmox_cluster_ status_info, while Deploy VM keeps running green	the runner's community.proxmox predates 2.0.0: Ubuntu's deb bundle (1.4.0) was silently serving everything, and it has every module the deploy path needs but not 08's cluster/node info modules; a user-path install is not enough if it lands outside the service user's real HOME (/home/semaphore, not /var/lib/semaphore)	sudo ansible-galaxy collection install community.proxmox:2.0.0 -p /usr/share/ansible/collections --force: the HOME-independent system path, visible to every runner context; the setup script now pins it there for rebuilds
Health Audit dies at "Read the node status": Requires proxmoxer 2.3 or newer; found version 2.2.0	community.proxmox 2.0.0's newer info modules declare a Python-client floor Ubuntu's python3-proxmoxer does not meet; the deploy path never noticed because proxmox_kvm/proxmox_vm_info declare no floor	sudo pip3 install --break-system-packages 'proxmoxer==2.3.0'; verify with python3 -c "import proxmoxer; print(proxmoxer.__version__)"; the pip copy in /usr/local shadows the apt one

Symptom	Cause	Fix
Health Audit task ends RED	by design: a CRITICAL or WARN finding exists; the full digest with every finding is in the task log above the verdict, and in the mail if armed	read the digest's FINDINGS section, fix what it names; the next audit clears the line. Pure-INFO runs (drift lines, new-since-last-audit) end green
Health Audit goes RED with AUDIT CRITICAL ... vanished since the last audit, for a VM you deleted on purpose	the baseline remembered the VM from the previous run, exactly its job; deleted-managed-VM is CRITICAL by design, so the task ends red and the subject carries CRITICAL	nothing to fix: confirm the deletion was yours; the baseline refreshed at the end of the red run already dropped the VM, and the next run is quiet (observed: CRITICAL run, then green 0/0/0)
The digest never arrives by mail	msmtp not installed/configured on the runner, or audit_mail_to is empty: the audit degrades to log-only and says so as an INFO line	run scripts/sem-01-msmtp.sh on sem-01 once, set audit_mail_from/audit_mail_to, re-run. Hand-rolled setups: msmtp treats an unopenable logfile as FATAL and the audit runs it as the semaphore user, so /var/log/msmtp.log must be owned by the service user
Health Audit goes red at "Mail the digest" with 535 5.7.8 Username and Password not accepted	the Gmail app password was revoked or rotated while mail was still armed; a configured-but-broken mail path is FATAL by design, and no digest is lost (it is printed above the mail step)	re-arm (new app password via the script) or retire gracefully: set audit_mail_to: "" for log-only mode. The documented order is retire-then-revoke, not the other way round
A second task sits WAITING while a long deploy runs; the evening digest arrives late	the task queue runs strictly one task at a time, cross-template included: a long deploy queues everything behind it, scheduled audits included	nothing to fix in the lab; in production, plan maintenance windows so scheduled audits are not starved
A run's pull phase prints "error: object file ... is empty" and "fetch-pack: invalid index-pack output", then re-clones and finishes green	a fetch into the template's cached clone was interrupted mid-write; Semaphore detects the unusable cache and falls back to a fresh clone	none needed, it already re-cloned; if it recurs on every run, delete the stale cache dir (repository_*_ under /var/lib/semaphore/tmp/) and let the next run re-clone

Table 8. The Phase 5 rows of the troubleshooting table, as they stand in the README today. The first three symptoms are the product working; the next two are machine state, not repo state; the rest are the mail lifecycle and the queue, observed.

One pattern deserves to be named because it recurred across phases: five of the field's bugs across the series were install-time or machine-state issues, a locale, a file ownership, a deb bundle, a Python floor, an unpushed commit, and only the last of those five was even in the repo's power to cause. The playbook code, sandbox-proven both times, shipped zero field bugs in Phase 5; every red run of 5b was the runner disagreeing with the repo about what is installed and where. That is the argument for pinning, in the setup script and in requirements.yml, and it is why the setup script's job description quietly grew from "install semaphore" to "rebuild sem-01 into the machine the playbooks expect."

9. Where the Project Stands

With Phase 5 closed, the loop the series set out to build is complete, and it is worth walking around it once. A request arrives through a typed form. The gate reads the live cluster and answers three questions, refusing cheaply when the answer is no. The pipeline clones, configures, mounts, agents, and delivers with a receipt. Twice a day, unattended, the watchman reads the same truth from the other side, remembers what live state cannot, and reports. The queue serializes the doors so the machine never argues with itself.

Run	What it was	What it closed
#38	adoption re-run, green	the unpushed-code lesson: a green run of old code proves nothing; three independent signatures (pull hash, task arithmetic, missing output)
#41	admission grant	Test A: live math 5/8 vCPU, 7/16 GB; the alpine fleet correction, the gate right where the prediction was wrong
08:12	collision refusal	Test B: owner named (VMID 103), changed=0, zero qm footprint
09:04	collision again, by accident	the misfiled Test C: assert priority proven, collision outranks squatter when both apply
09:05	budget refusal	Test D: 7/8 vCPU and 23/16 GB printed, RAM the trigger, ignored=1 fingerprint
09:12	squatter refusal	Test C re-run: probe answers at the runner's own IP, the squatter branch, ignored=0 fingerprint; 5a closed 4/4
#48, #50, #51	three red audits	the runner onion: deb-bundled collection 1.4.0, the HOME the task never reads, the proxmoxer 2.3 floor
#52	first green audit	baseline written, 5 INFO lines each a design promise, the baseline-path doc bug found in the digest itself
#53	second audit	drift-zero, byte-identical baseline rewrite: the idempotency proof

Run	What it was	What it closed
#55	deploy confidence re-run	the upgrades proven under the deploy path; deprecation warning gone; git self-heal and out-of-band reboot, both benign
#56	mail-proof audit	digest in the inbox, subject line exact; the mail chain closed end to end
09:10 UTC	scheduler-driven digest	the quiet fleet: 0/0/0, the empty-findings branch; 5b closed end to end
#58, #59	Round 0	same-template serialization: the second task read the first's committed state, refused at 9/8; the queue is an accidental mutex
#60, #61	cross-template probe	the audit waited out the deploy, per timestamps: global queue; first two-guest digest
#62	deleted-VM alarm	CRITICAL, red task, CRITICAL-subject mail: the baseline remembering what live state cannot; my INFO prediction miss, kept
#63	self-heal	0/0/0 quiet fleet, the vanished line gone; the mail step's 535 on the retired password, fatal by design; 5c closed

Table 9. The Phase 5 field ledger: every run that closed something, in order. Sixteen entries, five phases of the story, zero cleanup after any refusal.

The receipts rule, and the scoreboard's misses

The series' honesty contract says claims carry receipts, and Phase 5 had an unusual density of them because the operator pasted everything back in real time: the admission line with live math, four refusal transcripts with their recaps, three dead-task errors and their fixes, a digest in an inbox with a predicted subject, a scheduler-driven digest, a CRITICAL alarm and its self-heal. The same contract says the prediction scoreboard prints its misses, and this phase had three worth keeping. The fleet-math miss, 4/8 predicted against 5/8 real, was the design proving itself against its author. The severity miss, INFO predicted for a deleted VM that shipped as CRITICAL, was a misremembered severity table, and the field quoting the code back at the guide is the system working. The info-count miss on the mail-proof run, primary guess zero against three real, was the reboot detector being right about a reboot the predictor had forgotten. Three misses, three different causes, one lesson: the field does not read the docs before disagreeing with them.

The honest backlog

Nothing on this list blocks anything; it is the honest remainder after the roadmap's last row closed. TLS to the log receiver (port 6514) replaces the plain TCP forward when a real QRadar arrives. The resize button, the roadmap's old 4d item, is still two commands and a re-run away from being a template. Load testing beyond the single-node lab, high-availability for the runner itself, real licensed agent payloads behind the Phase 3 marker contract, and the AWX watch, a frozen project that would become interesting

again if it ever ships a release. The post-apply re-verification design from chapter 7 stays written down for the day the pipeline grows a CLI or API front door. And the portal design from chapter 1 stays parked, three curls and a Django skeleton away, if the customer ask ever returns.

The series, complete

Seven volumes now: the golden template, storage, the agents, the request form, this one, and the retrospective that analyzed the whole journey while it was still warm. The retrospective's numbers have aged by exactly one phase: the guide series it inventoried had five volumes and 112 pages, and this closing volume brings the set to six and roughly 145. The series' rule held to the last page: nothing was printed before it ran, every edition kept its errors, and the final phase's guide was written with the field logs still open in the other window. If a Phase 6 ever happens, the shape is already known from the backlog above; what the series proved is that the shape will change in the field, and that this is fine, as long as the receipts keep up.

0

cleanup runs after a refusal, zero half-built VMs across four refusal branches, and one queue that turned a race condition into a round of applause for the accident of serialization. The gate, the watchman, and the queue: Phase 5 closed the roadmap with nothing left unproven.